Carsharing Project Kamili Drive

CIM presentation

12.01.2026



Goal of the project



- Tanzania characterized by difficult conditions in current passenger transport
- service which brings passengers and drivers together
- an app and also a service via phone call/SMS
- generation of the profitable business for Tanzania and potentially neighbouring countries

Geography part - Building Bridges

- Little communication with Stakeholders
- Project Management
- survey optimization
- Geospatial Analysis

Progress: Communication issues

Last semester biggest problem: Communication.

We have established for the semester:

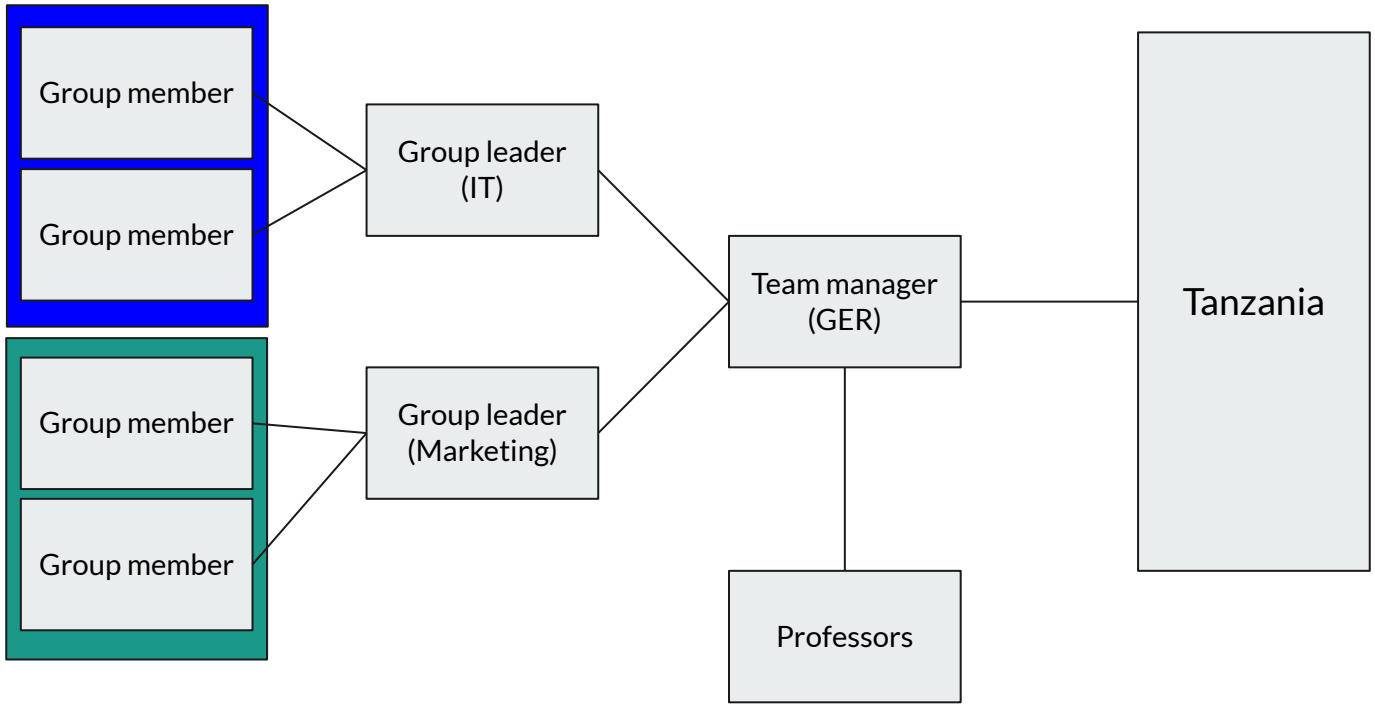
- Weekly subgroup meetings
- Monthly internal 5-minute-presentation on the subgroup progress in CIM HUB
- Soft-scale hierarchy / new group structure

We have planned for the next semester:

- Use of Project Management Tool
- Frequent contact to the Stakeholder

This semester only one non- technical student/Needed some time to implement new conduct

Progress: Communication issues



Project Management Tool

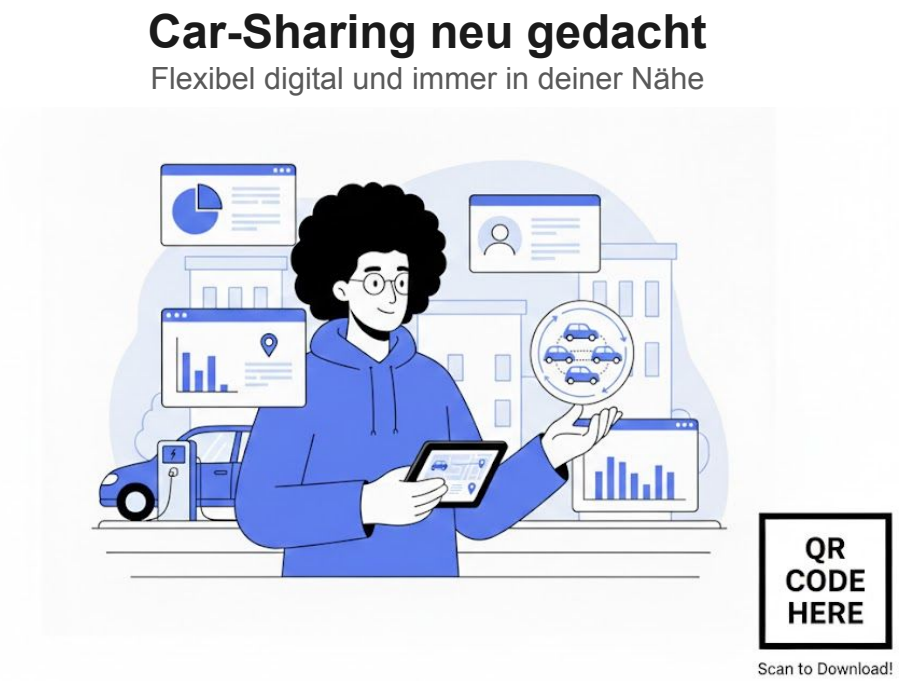
TANZANIA CAR-SHARING

EASY PROJECT MANAGEMENT

		PROJECT DETAILS							
TASK NAME	STATUS	PRIORITY	Start Date	Due Date	Timeline	Owner	DESCRIPTION	FILE	NOTE
Project Name									
	Not Started	High	1/8/2026	1/31/2026		Lionel	details of task here	uplaod file/ link	
	Complete	Medium	1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	
	Complete	Low	1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	
	Complete	High	1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	
Project Name									
	Needs Review		1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	
	Approved		1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	
	Overdue		1/1/2026	1/20/2026		NONE	details of task here	uplaod file/ link	

Progress: Survey optimization

- **"Join the Waiting List" flyers**
- **Incentivize participation**
- **Social media campaign**
- **Email existing networks**
- **Referral mechanism**
- **Targeted online ads .**



Potential of Using GIS for the Carsharing App

- Spatial Planning
- Accessibility Mapping
- Demand Visualization
- Infrastructure Assessment
- Route Optimization
- Strategic Decision-Making

Use of H3 Technology to create impactful and understable insights!

Technical - Bridging The Gap

1. The Problem (Last Semester)

- **The Situation:** We had a working backend and a mobile app, but they were separate.
- **The Issue:** The app used fake data. It looked complete, but you couldn't actually log in or save a trip to the database.

2. The Fix: Connecting It All

- **Our Focus:** We stopped building new features to focus on connecting the wires.
- **The Goal:** Replace the fake data with real connections. We needed to prove the phone could talk to the server.

3. Where We Are Now

- **Result:** We achieved our first successful working integration, but it still needs some improvements to be fully production-ready.

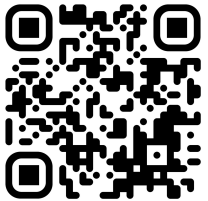
Backend: Technical Overview

Technology Stack & Architecture

- Architecture: Microservices with Spring Boot 3
- Runtime: Java 21
- Security: Spring Security (JWT-based authentication via Auth Service)
- Database: MongoDB Atlas (managed, scalable persistence)
- Location & Routing: OpenStreetMap and Google Maps API for geocoding, routing, distance/time estimation
- Containerization: Docker
- Service Discovery: Kubernetes-native discovery (DNS)

Core Services:

- API Gateway: Single entry point for all client requests
- Auth Service: User authentication and token validation
- User Service: Profile and account management
- Vehicle Service: Vehicle registration and tracking
- Trip Service: Trip planning and booking
- Ride Service: Active ride tracking



Backend: Key Terms & Definitions

Term	Definition	Key Point
User	A registered account in the system.	Has a role: **Driver**, **Passenger**, or **Admin**
Driver	A user who **creates trips** and is responsible for operating the vehicle.	Publishes trips + manages availability
Passenger	A user who **books seats** on trips offered by drivers.	Makes bookings and joins rides
Admin	A user with platform-level administrative permissions.	Moderation, audits, system management
Trip	A driver's full published journey from **origin → destination**.	Can contain multiple passenger ride segments
Booking	A passenger's reservation request for **seats on a trip**.	Request/hold/confirmed/cancelled lifecycle
Ride	A passenger's actual **segment** within a trip: **pickup → dropoff**.	Sub-route; may differ from trip origin/destination

Term	Definition
Total Seats	Maximum seats the Driver is offering on a Trip
Booked Seats	Number of seats currently reserved by Passengers
Requested Seats	Number of seats a Passenger requests in a Booking
Price Per Km	The cost rate set by the Driver for the Trip
Estimated Price	The calculated cost for a Passenger's Booking

Note: Admin role is not supported yet

Backend: Changes Summary

Platform & API Contract

- Upgraded runtime baseline from Java 17 → Java 21 (LTS)
- Enabled Dynamic Swagger / OpenAPI for easier frontend integration and maintainability
  - Added clearer model annotations (especially date-time fields)
- Standardized API responses across microservices
  - Unified success shape: ApiResponse
  - Unified error shape: ApiError (structured error body)
- Renamed confusing components for clarity (e.g., upcoming → active rides)
  - Bugfix: Empty responses for active bookings / trips (for passengers and drivers)

Infrastructure & Deployment

- Added Kubernetes integration + local Kind setup
- Migrated service discovery: Eureka → Kubernetes DNS
- Introduced a basic GitOps workflow (via Scripts)
- Deployed the microservices to GKE Cluster in GCP

```
hsamadov@Mac carsharing-backend % ./gitops/gitops.sh build-gcp auth-service user-service trip-service
🐳Building and pushing images to GCR...
📦Building selected services: auth-service user-service trip-service

🐳Build Configuration:
   Registry: eu.gcr.io
   Project: project-b05bd45b-70e8-4b7f-a65
   Tag: latest
   Services: auth-service user-service trip-service

🔑Configuring Docker for GCR...
```

Backend: Migration to Kubernetes

Eureka → Kubernetes DNS (what changed)

- Removed dependency on Eureka (no registry server to run/monitor)
- Services now discover each other via Kubernetes Services + DNS (e.g., auth-service.<namespace>.svc.cluster.local)
- Routing becomes Service-based instead of registry lookups

What Kubernetes unlocked

- Standard deployment model: same approach locally (Kind) and in cloud (GKE)
- Self-healing: automatic pod restarts, rescheduling, replica management
- Scalability: horizontal scaling via replicas / autoscaling patterns
- Built-in service networking: stable service endpoints + load balancing inside the cluster
- Operational consistency: liveness/readiness probes + health-based rollouts
- Safer rollouts: rolling updates, quick rollback with versioned manifests
- Better observability foundation: integrates cleanly with metrics/logging stacks (e.g., Grafana/Loki)

Result: Closer to a production-grade runtime with fewer moving parts and better reliability

Backend: GCP Pipeline (AR & GKE)

Name	Connection	Created	Updated
api-gateway	–	4 days ago	4 days ago
auth-service	–	4 days ago	4 days ago
trip-service	–	4 days ago	4 days ago
user-service	–	4 days ago	4 days ago
vehicle-service	–	4 days ago	4 days ago

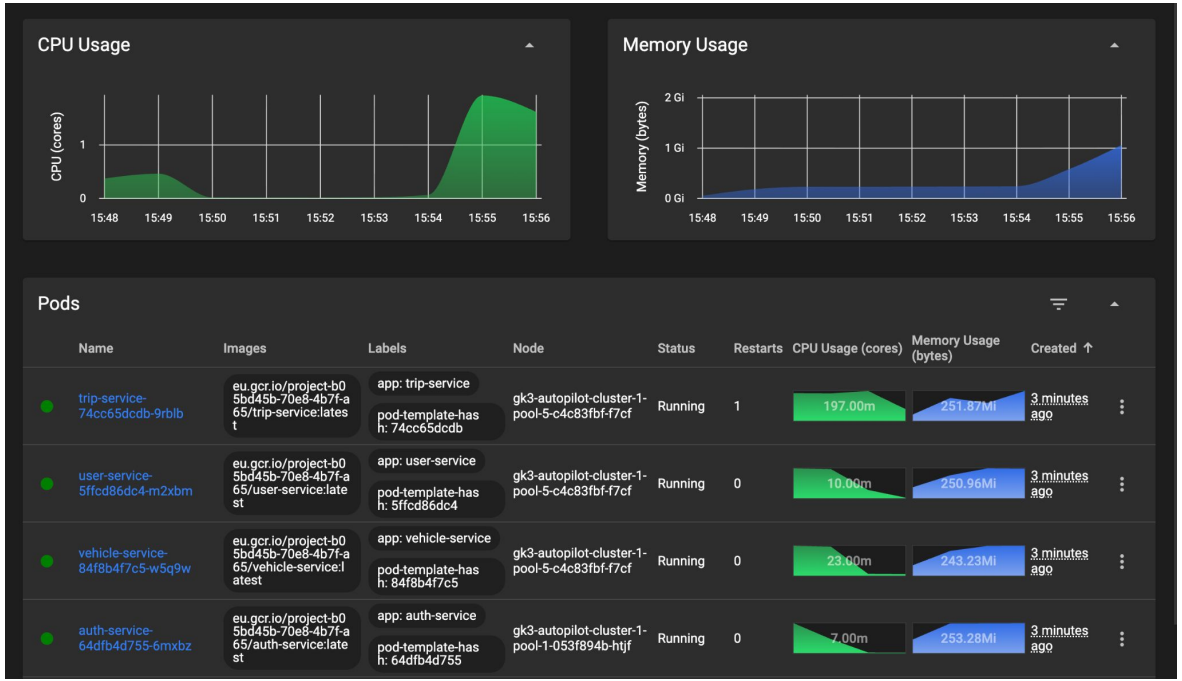
Container images are pushed to Artifact Registry.

Then they are deployed to GKE Cluster.

Status	Name	Location	Fleet	Number of nodes	Total vCPUs	Total memory	Notifications	Labels
✓	autopilot-cluster-1	europe-north1	carsharing-backend		1.5	5.25 GB	Set maintenance window	–

Name	Status	Type	Pods	Node type	Fleet	Namespace	Cluster
api-gateway	OK	Deployment	1/1	Autopilot-managed	carsharing-backend	carsharing	autopilot-cluster-1
auth-service	OK	Deployment	1/1	Autopilot-managed	carsharing-backend	carsharing	autopilot-cluster-1
trip-service	OK	Deployment	1/1	Autopilot-managed	carsharing-backend	carsharing	autopilot-cluster-1
user-service	OK	Deployment	1/1	Autopilot-managed	carsharing-backend	carsharing	autopilot-cluster-1
vehicle-service	OK	Deployment	1/1	Autopilot-managed	carsharing-backend	carsharing	autopilot-cluster-1

Backend: Kubernetes Dashboard



Live visibility: deployments, pods, rollouts

Resource usage monitoring (CPU/memory)

Quick post-deploy verification and troubleshooting workflow

Backend: Planned Improvements

Platform & API Quality

- Integrate Rate Limiting
- Add pagination for list/search endpoints (performance + predictable responses)

DB & Architecture Refinement

- Refine service boundaries for the Trip / Booking domain
    - make ownership clearer and reduce cross-service confusion
    - (Target: consistent terminology and cleaner responsibilities)
- Deciding: Migration from MongoDB to Postgres (because of PostGIS support - geospatial search)

Observability & Operations

- Add unit tests & CI
- Add centralized logging + metrics stack: e.g., Loki + Grafana
    - (includes dashboards for health, latency, errors)
- Improve runtime visibility:
    - request tracing (extend current request ID propagation)

Backend: Planned Features

Pricing

- MVP: driver sets a price when creating a trip
- Later: enforce dynamic min/max bounds based on route distance (based on price range per KM)

Trip & ride history

- history for both drivers and passengers (visibility and accountability)

Reviews & ratings

- trust and quality control for drivers and passengers

Event recording

- persist key domain events for debugging and analytics; can be viewed by Admin
- later extensible to a message bus (e.g., Kafka)

Notifications

- Booking and trip status updates in real time

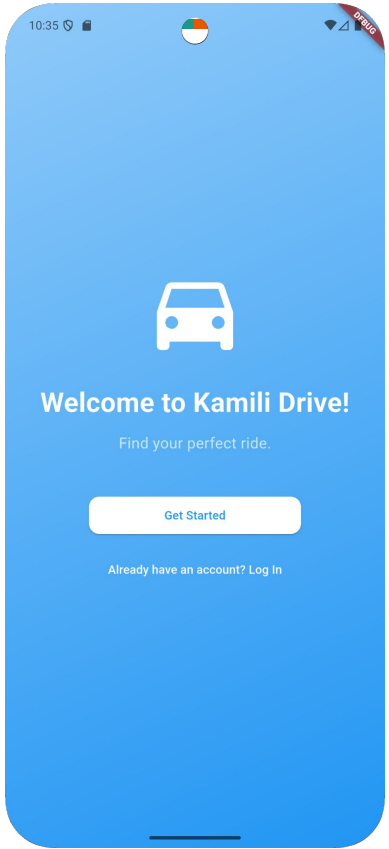
Front-End Side (Mobile Application)

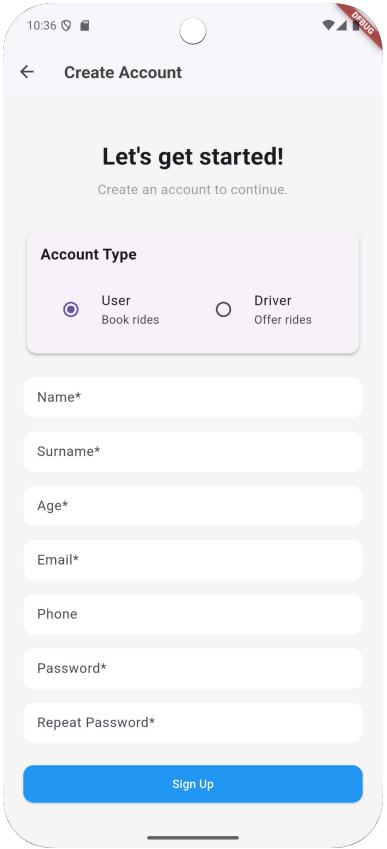
We implemented our mobile application using **Dart and the Flutter framework**, leveraging its cross-platform capabilities for a smooth development experience.

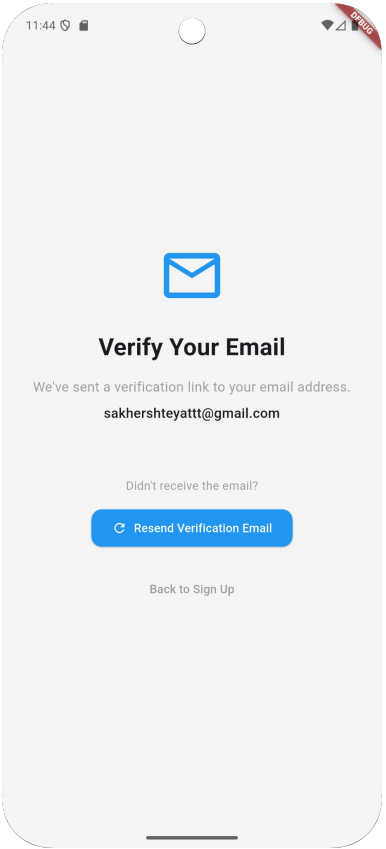
To support dynamic data handling during development, we integrated **Firebase** as a temporary backend solution.

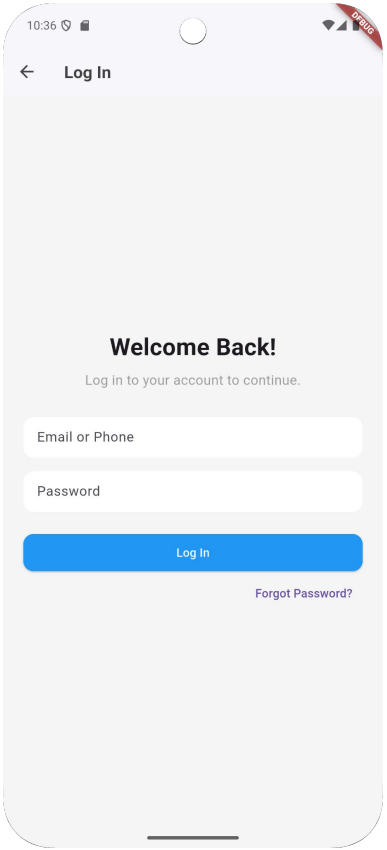
Key features and interface elements have been designed and are presented here to demonstrate our progress and user experience vision.

Welcome Page and Login/Sign-In Features

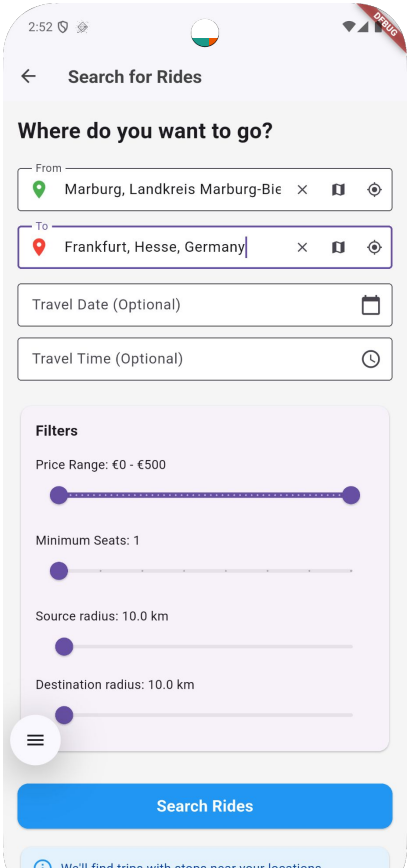


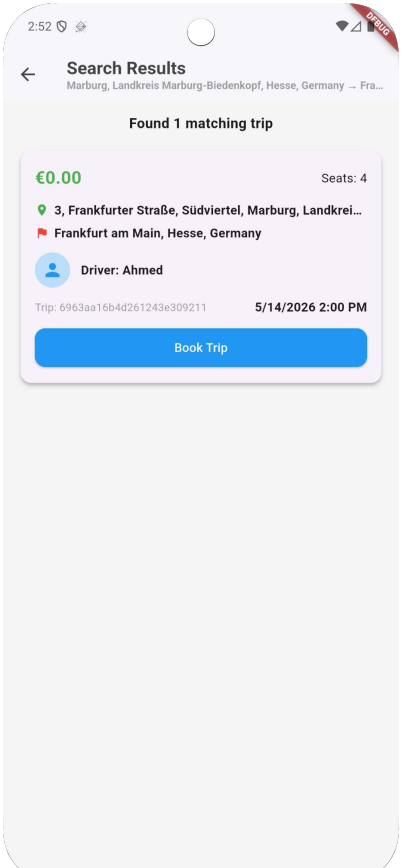


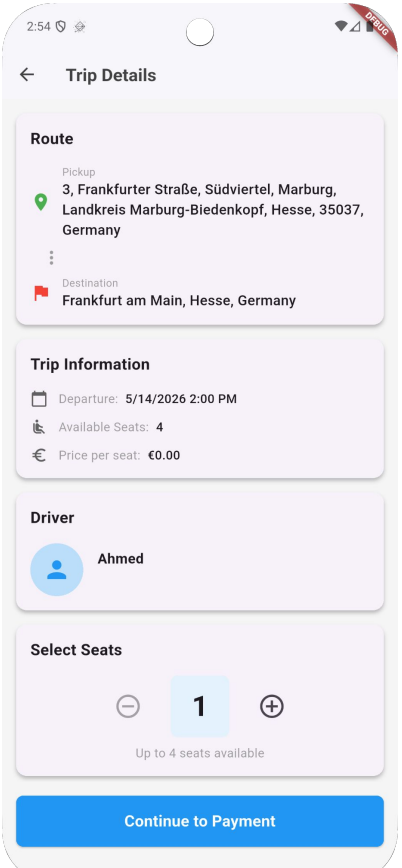


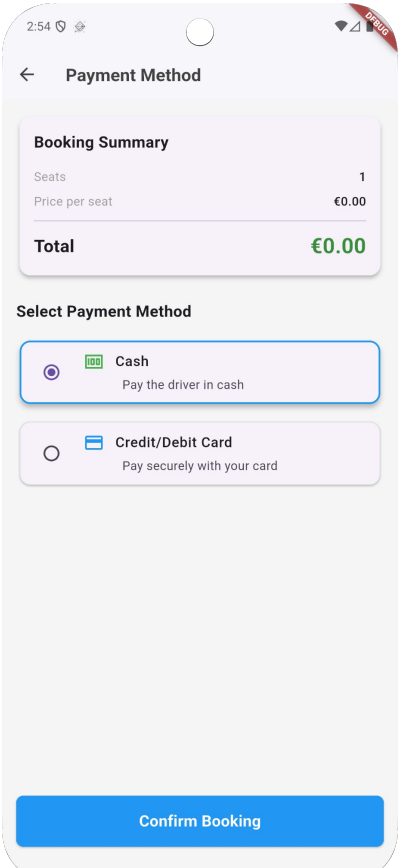


User pages

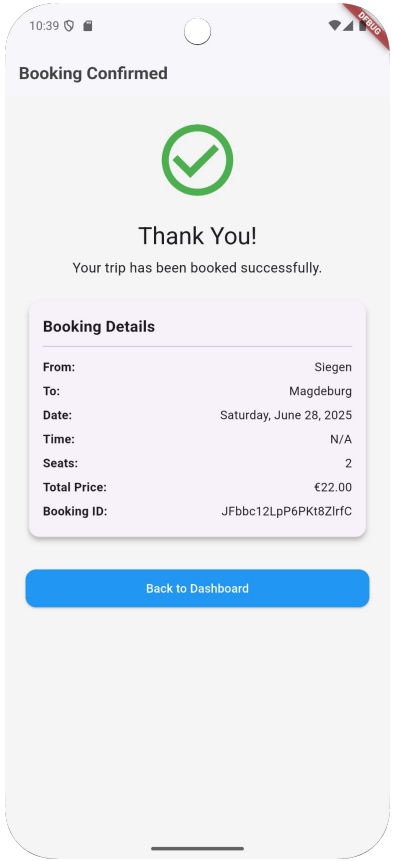

2:52
Search for Rides
Where do you want to go?
From
Marburg, Landkreis Marburg-Bie
To
Frankfurt, Hesse, Germany
Travel Date (Optional)
Travel Time (Optional)
Filters
Price Range: €0 - €500
Minimum Seats: 1
Source radius: 10.0 km
Destination radius: 10.0 km
Search Rides

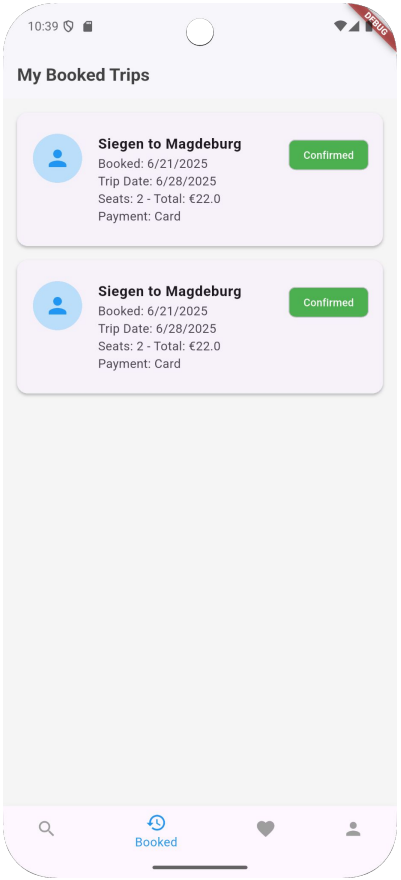

2:52
Search Results
Marburg, Landkreis Marburg-Biedenkopf, Hesse, Germany → Fra...
Found 1 matching trip
€0.00
Seats: 4
3, Frankfurter Straße, Südviertel, Marburg, Landkrei...
Frankfurt am Main, Hesse, Germany
Driver: Ahmed
Trip: 6963aa16b4d261243e309211
5/14/2026 2:00 PM
Book Trip

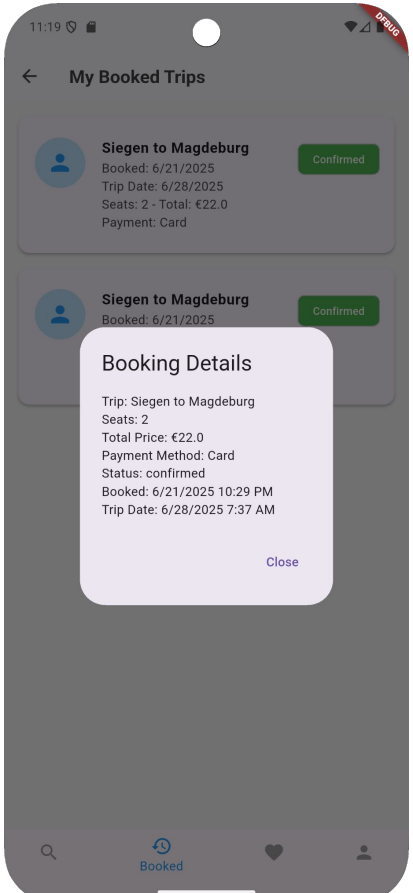

2:54
Trip Details
Route
Pickup
3, Frankfurter Straße, Südviertel, Marburg, Landkreis Marburg-Biedenkopf, Hesse, 35037, Germany
Destination
Frankfurt am Main, Hesse, Germany
Trip Information
Departure: 5/14/2026 2:00 PM
Available Seats: 4
Price per seat: €0.00
Driver
Ahmed
Select Seats
1
Up to 4 seats available
Continue to Payment


2:54
Payment Method
Booking Summary
Seats 1
Price per seat €0.00
Total €0.00
Select Payment Method
Cash
Pay the driver in cash
Credit/Debit Card
Pay securely with your card
Confirm Booking

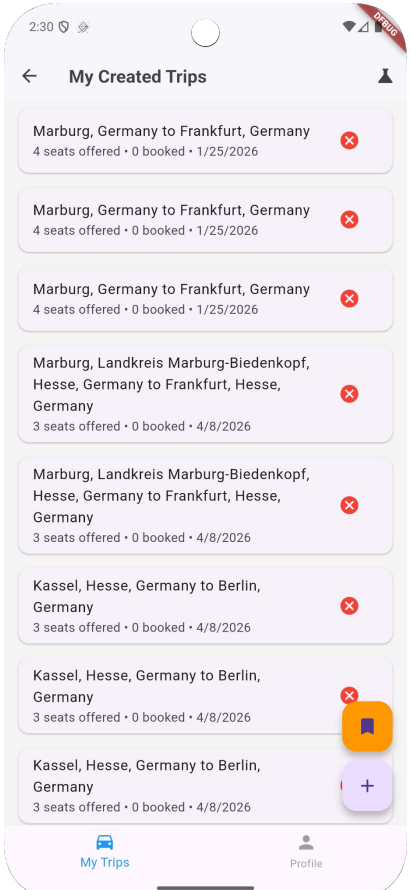
User pages

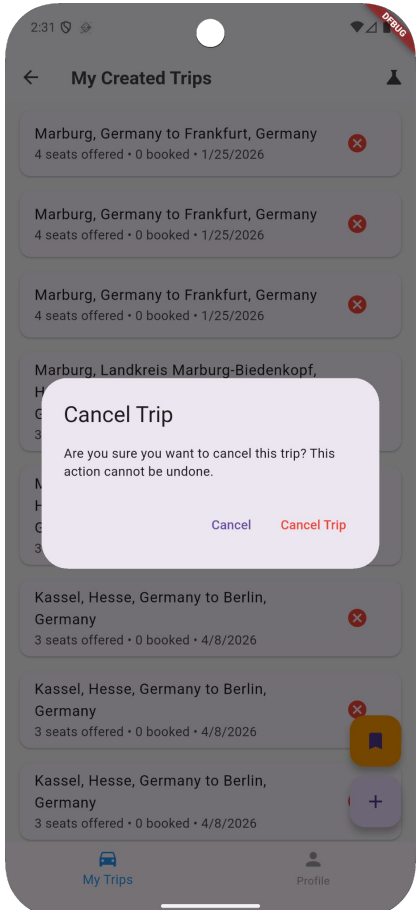

10:39
Booking Confirmed
Thank You!
Your trip has been booked successfully.
Booking Details
From: Siegen
To: Magdeburg
Date: Saturday, June 28, 2025
Time: N/A
Seats: 2
Total Price: €22.00
Booking ID: JFbbc12LpP6PKt8ZlrfC
Back to Dashboard

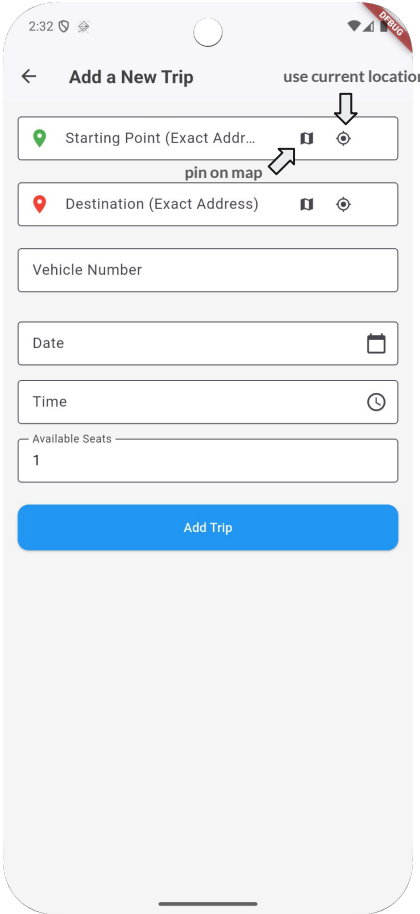

10:39
My Booked Trips
Siegen to Magdeburg
Booked: 6/21/2025
Trip Date: 6/28/2025
Seats: 2 - Total: €22.0
Payment: Card
Confirmed
Siegen to Magdeburg
Booked: 6/21/2025
Trip Date: 6/28/2025
Seats: 2 - Total: €22.0
Payment: Card
Confirmed
Booked


11:19
My Booked Trips
Siegen to Magdeburg
Booked: 6/21/2025
Trip Date: 6/28/2025
Seats: 2 - Total: €22.0
Payment: Card
Confirmed
Booking Details
Trip: Siegen to Magdeburg
Seats: 2
Total Price: €22.0
Payment Method: Card
Status: confirmed
Booked: 6/21/2025 10:29 PM
Trip Date: 6/28/2025 7:37 AM
Close
Booked

Driver pages









Architecture Upgrades:

REST API Migration: Trips, bookings.

Maps Integration: Interactive OpenStreetMap with markers, routes, GPS.

State Management: Riverpod providers, clean repository pattern.

Network Layer: Dio with interceptors, JWT auth, secure storage.

Serialization: json_serializable for fast, type-safe parsing.

Features & UX

Booking Flow: Details → payment → confirmation.

Advanced Search: Date/time/radius filters with map visualization.

User Dashboard: Booked trips with pull-to-refresh and local caching.

Driver Dashboard: Passenger names and booking counts per trip.

UI/UX: Material 3 design, status badges, clear CTAs, error handling.

Quality, Security

Bug Fixes: Widget lifecycle errors, 400 search params.

Performance: Provider caching time-out, efficient JSON, optimized map tiles.

Security: JWT auth tokens, flutter_secure_storage, auth interceptors.

Migration Challenges

Schema Mismatches: Aligning field names and payloads (e.g., lat/lon vs sourceLat/sourceLon, response wrappers like data).

Auth & Tokens: Switching from Firebase Auth to JWT, handling headers, refresh, and 401 flows.

Consistency & Visibility: Bookings created via API not appearing in list endpoints; eventual consistency vs immediate reads.

Error Semantics: Handling 409 ALREADY_BOOKED, clearer server-side validations, and uniform error structures.

Search Behavior: Exact date/time requirement vs flexible windows; reconciling filtering logic and time zones.

Data Ownership: Moving user profile lookups to Firebase while core trip/booking logic lives in REST API.

Logging & Observability: Needing richer backend logs to debug parsing, empty lists, and edge cases.

Semester Roadmap:

1: User Authentication

Backend Status: Core APIs are functional, including Registration & Login

Frontend Status: Basic Mobile UI connected and verified.

- **Future Enhancements:** Implement verification layers for Email and Phone Numbers to increase trust

.2: Driver & Vehicle Onboarding

- **Frontend Status:** 2-step Mobile UI implementation is ready (Driver Registration & Vehicle Addition).
- **Backend Action:** Refactoring driver registration and vehicle APIs to align with the new 2-step mobile flow.

Semester Roadmap:

3: Core Ride Marketplace & Matching Logic

- **Goal:** Implement the complete booking lifecycle where drivers publish trips using **Maps integration** for precise location tracking, and passengers are matched to these rides via a **proximity-based algorithm** (filtering by time and radius) that requires manual **driver approval** to finalize the booking.
- Drivers create a trip: starting point, and finish point. Also adds possible stops
- The user searches for a ride: They choose their start and destination points.
  We check the possible ride stops matching the timeframe they want, within the radius.
- **Ride Confirmation by Driver**
- Possible enhancement for driver: put previous rides as a template
- Possible enhancement for user: Seat Selection

4: Review System for User & Driver

- After the ride finishes the user & driver rate each other



Thank you for your attention!